Task 4:

In this task we added unary minus to LangVar by extending it into LangUnary. This lets us write things like print(-5);, print(-x); and print(-(2+3)); instead of always subtracting from zero.

We kept the LangVar grammar and added an expr_unary rule for the minus sign. We placed it before multiplication and division, so an expression like -2 * 3 is understood as (-2) * 3, just like in C. The main rule is:
?expr_unary: INT64_MIN_LITERAL -> expr_int64_min
| "-" atom -> expr_neg
| atom

By using atom after the minus, we made sure print(--1); is not allowed. But print(1 - -1); and print(-(-1)); are allowed. We also adjusted the number tokens so they do not include the minus sign themselves.

We added a NegOp node to the AST to store what needs to be negated. The interpreter calculates that expression first, then changes its sign. For example, print(-(2+3)); gives -5. We also updated the pretty printer for the new expression.

To test it, we added 13 programs in progs/unary and 12 unit tests. They passed locally together with the original unary program. The full ./check.sh still needs to be run in the final setup.